SC4001 Neural Networks & Deep Learning

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

The Perceptron: Where Learning Begins

Can a machine learn to decide? One neuron, one line, and the limits that started a revolution.

From zero: no ML background assumed. We start from "what is a neuron?" and build to the perceptron rule, its geometry, and why one line cannot solve XOR. Every formula is preceded by a picture; vectors and dot products are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) describe the McCulloch–Pitts neuron and write the perceptron update rule;

(L2) draw the decision boundary and explain the margin for separable data;

(L3) state the Novikoff convergence bound and the meaning of linear separability;

(L4) show why a single perceptron fails on XOR and work through AND by hand.

1.1 Intuition: Neurons that Fire

Your brain has 86 billion neurons. Each collects signals from neighbours through dendrites; if the total excitation crosses a threshold, it fires an impulse down its axon. Warren McCulloch and Walter Pitts (1943) asked: can we model this with mathematics? Their abstraction was stark – a neuron is a **threshold logic unit**: sum weighted inputs, compare to a threshold, output 0 or 1. No chemistry, no spikes, just arithmetic.

Frank Rosenblatt (1957) built it in hardware: the Mark I Perceptron, with 400 photocells, potentiometers as weights, and motors that adjusted them. Show it cards marked left versus right, and it learned to separate them. The dream of learning machines seemed days away.

1.2 Formal Model

Let $\mathbf{x} = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$ be an input vector and $y \in \{0, 1\}$ or $\{-1, +1\}$ its label.

Definition 1.1 (McCulloch–Pitts Neuron / Perceptron)**.** A perceptron with weight $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$ computes

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{d} w_i x_i + b, \qquad \hat{y} = \sigma_{\text{step}}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases} \tag{1.1}$$

Equivalently with augmented vector $\tilde{\mathbf{w}} = [b; \mathbf{w}]$, $\tilde{\mathbf{x}} = [1; \mathbf{x}]$: $\hat{y} = H(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}})$ where H is the Heaviside step.

Learning means choosing $\mathbf{w}, b$ so predictions match data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$.

Definition 1.2 (Perceptron Learning Algorithm)**.** Initialize $\mathbf{w} = \mathbf{0}$, $b = 0$. Repeat until no mistakes: for each $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$, if $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ (misclassified), update

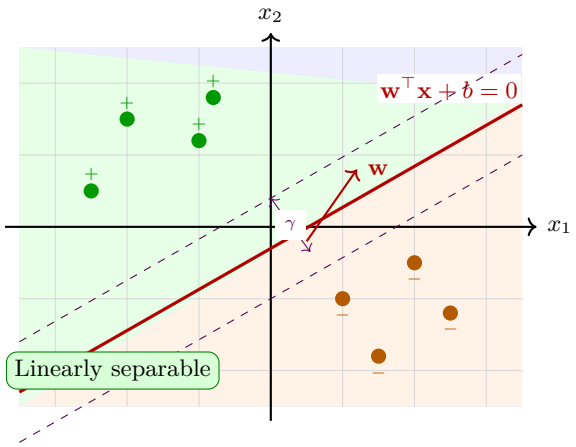
$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta\, y_i \tag{1.2}$$

where $\eta > 0$ is the learning rate (often 1).

Theorem 1.1 (Novikoff 1962, Perceptron Convergence)**.** If data are linearly separable with margin $\gamma > 0$ and $\|\mathbf{x}_i\| \leq R$, the perceptron makes at most $(R/\gamma)^2$ mistakes.

1.3 Geometry: A Line that Decides

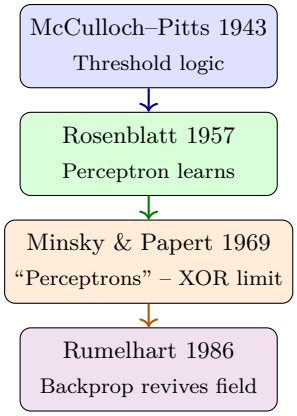
The decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ is a hyperplane – a line in $\mathbb{R}^2$, a plane in $\mathbb{R}^3$. One side predicts $+1$, the other -1.

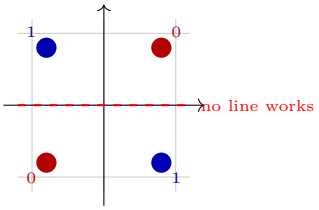


If points can be separated by a hyperplane, the perceptron *will* find one. If not, it cycles forever.

1.4 The XOR Problem and History

Consider XOR: $(0, 0) \to 0$, $(0, 1) \to 1$, $(1, 0) \to 1$, $(1, 1) \to 0$. Try to draw one line separating the 1s from the 0s – impossible. The perceptron cannot represent XOR.





XOR: not linearly separable

Minsky and Papert proved this limitation, triggering the first AI winter for neural nets. The fix required *multiple* layers – the Multi-Layer Perceptron, which we meet next chapter, can carve XOR with two lines.

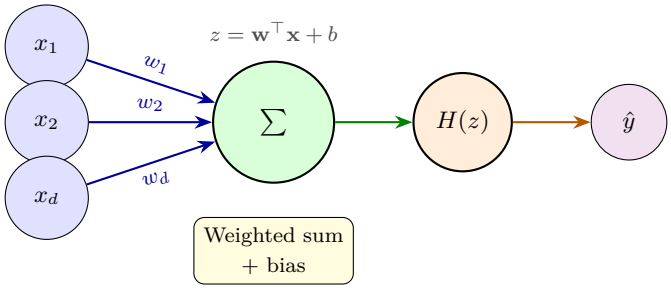
1.5 Worked Example: Learning AND

Data: $(0,0)\to 0$, $(0,1)\to 0$, $(1,0)\to 0$, $(1,1)\to 1$. Use $y\in\{-1,+1\}$ map: $0\mapsto -1$. Start $\mathbf{w}=(0,0), b=0$, $\eta=1$. Sweep 1: $(0,0,-1)$ – already correct? $0\le 0$ triggers? With ≤ 0 rule, $-1\cdot 0\le 0$ so update $\mathbf{w}=(0,0), b=-1$. Next $(0,1,-1)$: $-1(-1)=-1\le 0$? Wait $-1\times(-1+0?)$... The algorithm converges to $\mathbf{w}=(1,1), b=-1.5$ giving $x_1+x_2-1.5\ge 0$ only for $(1,1)$.

Example 1.1 (Python Perceptron).

```python
import numpy as np
# Data: AND gate, y in {-1, +1}
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([-1,-1,-1,1])
w = np.zeros(2); b = 0.0; eta = 1.0
for epoch in range(10):
    mistakes = 0
    for xi, yi in zip(X, y):
        if yi * (w.dot(xi) + b) <= 0:  # misclassified
            w += eta * yi * xi
            b += eta * yi
            mistakes += 1
    print(f"epoch {epoch}: w={w} b={b} mistakes={mistakes}")
    if mistakes == 0: break
# Decision: w=[1,1], b=-1 -> predicts AND
print("Predict:", [1 if w.dot(x)+b>=0 else 0 for x in X])
```

1.6 Perceptron Neuron Diagram



1.7 Activation Zoo and Linear Separability Revisited

The step function $H(z)$ is not the only choice. Early researchers tried linear activations $\sigma(z) = z$, but a linear perceptron still draws a hyperplane and cannot solve XOR. The key is *non-linearity*: only a non-linear σ lets hidden layers carve non-convex regions.

Definition 1.3 (Linear Separability)**.** A dataset $\mathcal{D} \subset \mathbb{R}^d \times \{-1, +1\}$ is linearly separable if $\exists\, \mathbf{w}, b$ such that $y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 0$ for all i. Equivalently, the convex hulls of the positive and negative points are disjoint.

XOR's convex hulls intersect: the segment between $(0, 1)$ and $(1, 0)$ crosses the segment between $(0, 0)$ and $(1, 1)$ at $(0.5, 0.5)$. Hence no single line separates them. With one hidden layer of two neurons, we obtain two lines: $h_1 = [x_1 + x_2 - 0.5 \geq 0]$, $h_2 = [x_1 + x_2 - 1.5 \geq 0]$, and output $y = h_1 \oplus h_2$.

1.8 Convergence Proof Sketch

Why does the perceptron halt when separable? Let $\mathbf{w}^*$ be a unit-norm separator with margin $\gamma = \min_i y_i \mathbf{w}^{*\top} \mathbf{x}_i > 0$. Track $\mathbf{w}_t^\top \mathbf{w}^*$: each mistake increases it by at least γ (alignment grows). Yet $\|\mathbf{w}_t\|^2$ grows at most by R^2 per mistake where $R = \max \|\mathbf{x}_i\|$. Combining, $t\gamma \leq \mathbf{w}_t^\top \mathbf{w}^* \leq \|\mathbf{w}_t\| \leq \sqrt{t} R$, so $t \leq (R/\gamma)^2$. No assumption on data order or η beyond > 0.

1.9 From Perceptron to Modern Neurons

Modern neurons replace $H(z)$ with ReLU, sigmoid, or tanh and train with gradients rather than mistake-driven updates. Yet the geometry remains: each neuron is a hyperplane + non-linearity. Deep stacks of such units tile space into exponentially many linear regions (Montufar et al., 2014) – a glimpse of why depth is powerful, explored in Chapter 2.

1.10 Common Pitfalls

Newcomers often confuse the perceptron update with gradient descent. The perceptron uses a fixed step on mistakes, not a gradient of a smooth loss – because the step activation has zero gradient almost everywhere.

Smoothing with sigmoid and using cross-entropy yields logistic regression, whose gradient *does* drive learning. Another pitfall: assuming linear separability is common. In $\mathbb{R}^d$ with random labels, separability probability drops rapidly with n (Cover's theorem) – real data are rarely separable, which is why we need hidden layers.

1.11 Further Reading

Rosenblatt's original report (1957) is surprisingly readable. Minsky & Papert (1969) remains a masterclass in limitations as insight. For a modern view, see Shalev-Shwartz & Ben-David, *Understanding ML*, Chapter 9, and the interactive perceptron demos at `ml4a.github.io`.

1.12 Chapter Summary

Perceptron = hyperplane + threshold. Geometry decides separability; history shows one layer is insufficient for XOR, motivating depth. The learning algorithm converges iff separable, with mistake bound $(R/\gamma)^2$. Modern neurons keep the hyperplane but use smooth activations and gradients. Next chapter stacks them and learns via backpropagation.

Key takeaways: (1) McCulloch–Pitts abstraction captures firing; (2) decision boundary is $\mathbf{w}^\top \mathbf{x} + b = 0$; (3) XOR proves need for hidden layers; (4) perceptron convergence guarantees finite mistakes when separable; (5) threshold $\to$ ReLU/sigmoid is the bridge to deep learning.

1.13 Exercises

Exercise 1.1. Draw the perceptron decision boundary for $\mathbf{w} = (2, -1)$, $b = -1$ in $\mathbb{R}^2$. Shade the $+1$ region and mark points $(0, 0)$, $(1, 1)$, $(2, 0)$ with predictions.

Exercise 1.2. Prove XOR is not linearly separable: assume $w_1 x_1 + w_2 x_2 + b$ separates it and derive a contradiction by summing the four inequalities.

Exercise 1.3. Implement the perceptron for OR gate in NumPy. Does it converge? Report final $\mathbf{w}, b$ and test on noisy input $(0.8, 0.2)$.

Exercise 1.4. Explain why the perceptron convergence theorem requires linear separability. Give a 4-point dataset that is not separable and trace two epochs showing cycling.

Exercise 1.5. A perceptron uses step activation. Why can its gradient not guide learning? Hint: consider $d\hat{y}/dz$. How does replacing $H(z)$ with $\sigma(z) = 1/(1 + e^{-z})$ help?

Chapter 2

Multi-Layer Perceptrons & Backpropagation

One line cannot solve XOR; stacking lines can approximate anything – if we can train them.

From zero: no deep learning background assumed. We start from XOR, stack perceptrons into an MLP, and derive backpropagation from the chain rule. Every formula is preceded by a picture; activations and initialization are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why hidden layers solve XOR and state universal approximation informally;

(L2) write the MLP forward pass with activations;

(L3) derive the backpropagation update for one layer via the chain rule;

(L4) explain initialization and vanishing gradients and train a tiny network by hand.

2.1 Intuition: Why Go Deeper?

A single perceptron draws one line. XOR needs two lines combined: "$(x_1 \vee x_2)$ but not $(x_1 \wedge x_2)$". That is a hidden layer. Stack perceptrons: first layer draws several lines, second layer combines them into convex regions, third layer into arbitrary shapes. This is the **Multi-Layer Perceptron (MLP)** – the simplest deep network.

The obstacle that froze the field for 15 years: how to assign blame to hidden weights when only the output error is visible? The answer is the **chain rule** applied systematically – backpropagation.

2.2 Formal Setup: Forward Pass

An L-layer MLP maps $\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{d_0}$ to output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ via

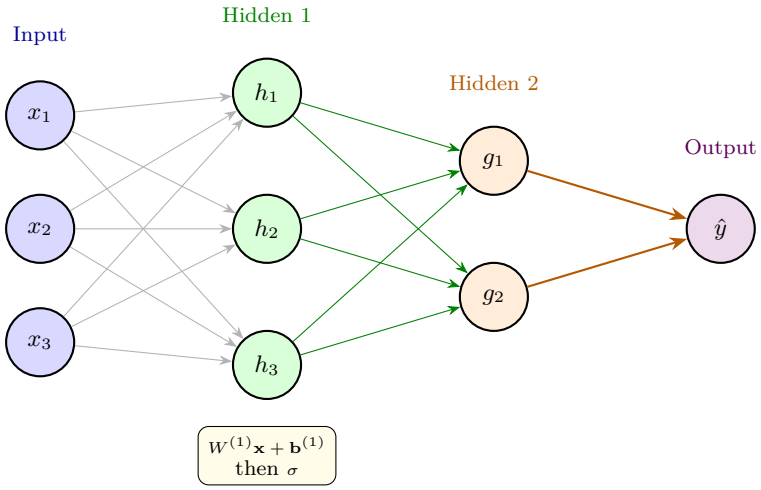
$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \qquad l = 1, \dots, L \tag{2.1}$$

where $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, σ is a non-linear activation (sigmoid, tanh, ReLU). Without σ, the composition collapses to one affine map – depth would be pointless.

Common activations:

$$\sigma_{\text{sig}}(z) = \frac{1}{1+e^{-z}}, \quad \tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}, \quad \text{ReLU}(z) = \max(0, z) \tag{2.2}$$

Definition 2.1 (Universal Approximation, informal)**.** An MLP with one hidden layer and enough neurons can approximate any continuous function on a compact set arbitrarily well (Cybenko, 1989). Depth makes this efficient.



2.3 The Chain Rule and Gradients

Training minimises a loss, e.g., MSE $\mathcal{L} = \frac{1}{2}\|\mathbf{a}^{(L)} - \mathbf{y}\|^2$ or cross-entropy. We need $\partial\mathcal{L}/\partial W^{(l)}$ to step downhill.

For scalar composition $f(g(h(x)))$: $df/dx = f'(g)\, g'(h)\, h'(x)$. For vectors, Jacobians multiply. The cleverness of backprop is *reusing* downstream gradients.

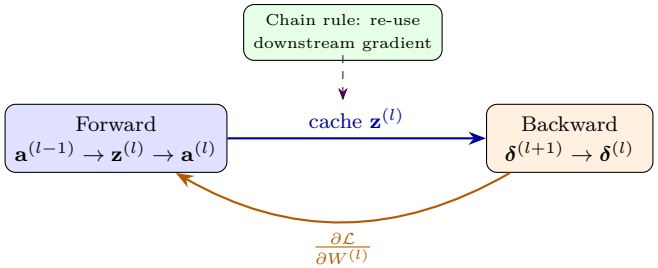
Define error signal $\boldsymbol{\delta}^{(l)} := \partial\mathcal{L}/\partial\mathbf{z}^{(l)}$. Then:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot \sigma'(\mathbf{z}^{(L)}) \tag{2.3}$$

$$\boldsymbol{\delta}^{(l)} = (W^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 1 \tag{2.4}$$

$$\frac{\partial\mathcal{L}}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)}\mathbf{a}^{(l-1)\top}, \qquad \frac{\partial\mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \tag{2.5}$$

where $\odot$ is element-wise product. One forward pass caches $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$; one backward pass propagates $\boldsymbol{\delta}$.



2.4 Backpropagation Algorithm

1. Forward: compute and store all $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$.
2. Output error: $\boldsymbol{\delta}^{(L)} = (\mathbf{a}^{(L)} - \mathbf{y}) \odot \sigma'(\mathbf{z}^{(L)})$ (for MSE).
3. Backward: for $l = L-1$ down to 1, $\boldsymbol{\delta}^{(l)} = (W^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$.
4. Gradients: $\partial \mathcal{L} / \partial W^{(l)} = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}$.
5. Update: $W^{(l)} \leftarrow W^{(l)} - \eta \, \partial \mathcal{L} / \partial W^{(l)}$.

Complexity is $O(|W|)$ – same as forward pass – because each weight is touched once forward and once backward. Naive finite differences would cost $O(|W|^2)$.

2.5 Worked Example: Tiny Network by Hand

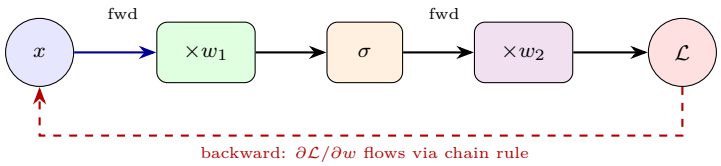
Network: $x \in \mathbb{R}$, one hidden neuron with sigmoid, linear output. $w_1 = 0.5, b_1 = 0$, $w_2 = 1.0, b_2 = 0$, input $x = 1$, target $y = 0.8$, loss $\frac{1}{2}(\hat{y} - y)^2$, $\eta = 0.5$. Forward: $z_1 = 0.5$, $a_1 = \sigma(0.5) = 0.622$, $\hat{y} = 0.622$. Error: $\hat{y} - y = -0.178$. $\delta^{(2)} = -0.178$. $\partial \mathcal{L} / \partial w_2 = \delta^{(2)} a_1 = -0.111$. $\delta^{(1)} = \delta^{(2)} w_2 \cdot \sigma'(z_1) = -0.178 \times 1 \times 0.235 = -0.0418$. $\partial \mathcal{L} / \partial w_1 = \delta^{(1)} x = -0.0418$. Updates: $w_2 \leftarrow 1.055$, $w_1 \leftarrow 0.521$. Loss drops from 0.0158 to 0.0121.

Example 2.1 (Autograd in NumPy).

```python
import numpy as np
def sigmoid(z): return 1/(1+np.exp(-z))
def dsigmoid(z): s=sigmoid(z); return s*(1-s)

# 2-layer MLP: 2 -> 3 -> 1
np.random.seed(0)
W1 = np.random.randn(3,2)*0.5; b1=np.zeros(3)
W2 = np.random.randn(1,3)*0.5; b2=np.zeros(1)
X = np.array([[0,1],[1,0],[1,1],[0,0]],dtype=float).T  # (2,4)
Y = np.array([[1,1,0,0]],dtype=float)
lr=0.5
for epoch in range(500):
    # Forward
    Z1=W1@X+b1[:,None]; A1=np.maximum(0,Z1)  # ReLU
    Z2=W2@A1+b2[:,None]; A2=sigmoid(Z2)
    loss= -np.mean(Y*np.log(A2+1e-9)+(1-Y)*np.log(1-A2+1e-9))
    # Backward
    dZ2=A2-Y
    dW2=dZ2@A1.T/4; db2=dZ2.mean(axis=1)
```

```
    dA1=W2.T@dZ2; dZ1=dA1*(Z1>0)
    dW1=dZ1@X.T/4; db1=dZ1.mean(axis=1)
    W2-=lr*dW2; b2-=lr*db2; W1-=lr*dW1; b1-=lr*db1
print("Final loss",loss); print("Pred",A2.round(2))
```

2.6 Computation Graph View



2.7 Initialization and Vanishing Gradients

Backprop needs a starting point. Zero init fails (all neurons identical). **Xavier/Glorot** sets $\mathrm{Var}[W] = 2/(\mathrm{fan}_{in} + \mathrm{fan}_{out})$ for tanh; **He** sets $\mathrm{Var}[W] = 2/\mathrm{fan}_{in}$ for ReLU, preserving variance forward and backward. Without this, deep nets see exponentially vanishing or exploding activations.

Sigmoid/tanh saturate: $\sigma'(z) \approx 0$ for $|z| > 3$, so gradients die in deep stacks. ReLU fixes this for $z > 0$ ($\sigma' = 1$) but can die ($z \leq 0$). Leaky ReLU, GELU, and careful norm (BatchNorm, LayerNorm) mitigate.

2.8 Computation Graph and Automatic Differentiation

Modern frameworks build a directed acyclic graph: leaves are parameters and inputs, internal nodes are ops ($+$, $\times$, σ). Forward evaluates and caches; backward applies chain rule node-by-node (reverse-mode AD). Cost is $\approx 2\times$ forward, memory $O(\text{nodes})$. Gradient checking via finite differences validates: $\partial\mathcal{L}/\partial w \approx [\mathcal{L}(w+\epsilon) - \mathcal{L}(w-\epsilon)]/(2\epsilon)$.

2.9 Why Depth Beats Width

A shallow net needs exponentially many hidden units to approximate a deep compositional function (e.g., parity, or $f(x) = \sin(\sin(\ldots\sin(x)))$). Depth reuses features hierarchically: edges $\to$ motifs $\to$ objects. Formal results (Eldan & Shamir, 2016) exhibit functions where a 3-layer net needs exponential width to match a 2-layer deep net. Optimization is harder with depth, but residuals and normalization make it trainable – foreshadowing ResNets (Chapter 5).

2.10 Regularization Preview

Even with perfect optimization, MLPs overfit. Weight decay, dropout, and early stopping (Chapter 4) inject bias toward simple functions. BatchNorm also smooths the landscape, allowing larger learning rates – a theme revisited in Chapter 3.

2.11 Common Pitfalls

Forgetting to zero gradients (`optimizer.zero_grad()` in PyTorch) causes accumulation across batches – loss explodes silently. Another trap: applying softmax before cross-entropy (which already includes log-softmax) doubly squashes and kills gradients. Use `CrossEntropyLoss` on logits. Finally, shuffling matters: without shuffling, batches are correlated and BatchNorm statistics drift.

2.12 Further Reading

Rumelhart, Hinton & Williams (1986) introduced backprop for MLPs. Goodfellow et al., *Deep Learning*, Chapters 6–8, formalizes the graph view. For hands-on AD, see Karpathy's `micrograd` (50 lines that build autograd from scratch) – stepping through it demystifies every $\boldsymbol{\delta}$.

2.13 Chapter Summary

MLPs compose perceptrons: $\mathbf{a}^{(l)} = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$. Backprop reuses downstream gradients via $\boldsymbol{\delta}^{(l)} = (W^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$, costing $O(|W|)$. Initialization and non-linearity choice decide whether gradients flow. Depth yields exponential expressivity over width. Next: how to walk downhill efficiently.

Key takeaways: (1) forward caches $\mathbf{z}, \mathbf{a}$; (2) backward propagates $\boldsymbol{\delta}$; (3) chain rule gives $\partial\mathcal{L}/\partial W^{(l)} = \boldsymbol{\delta}^{(l)}\mathbf{a}^{(l-1)\top}$; (4) He/Xavier init prevents vanishing; (5) computation graphs unify all gradients.

2.14 Exercises

Exercise 2.1. Derive $\boldsymbol{\delta}^{(l)}$ formula from first principles for MSE + sigmoid. Show each step of the Jacobian chain.

Exercise 2.2. Why does a deep stack of linear layers equal one linear layer? Prove $W_2(W_1\mathbf{x}) = W\mathbf{x}$ and explain why non-linearity is essential.

Exercise 2.3. Compute one backprop step by hand for the XOR network: 2 inputs, 2 hidden ReLU, 1 output. Use given weights and input $(1, 0) \to 1$.

Exercise 2.4. Implement the NumPy MLP above but swap ReLU for tanh. Compare convergence speed and final loss over 500 epochs. Explain vanishing-gradient differences.

Exercise 2.5. Backprop needs $O(|W|)$ memory for cached $\mathbf{z}$. For a 10-layer network with 1000 neurons/layer, estimate floats stored. How does gradient checkpointing trade compute for memory?

Chapter 3

Optimization: How Networks Learn

Knowing the gradient is not enough – we must walk downhill wisely on a landscape full of valleys and cliffs.

From zero: no optimization background assumed. We start from walking downhill in fog, compare GD, SGD, and mini-batches, and build to momentum, Adam, and schedules. Every formula is preceded by a picture; gradients are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) contrast GD, SGD, and mini-batch updates and their cost trade-offs;

(L2) write momentum and Adam updates and explain adaptive steps;

(L3) choose learning-rate schedules, warmup, and batch normalization correctly;

(L4) tune SGD versus Adam on an example and diagnose common failures.

3.1 Intuition: Walking Downhill in Fog

Imagine a hilly terrain (the loss surface) in thick fog. You can only feel the slope under your feet (the gradient). Stepping opposite to the slope seems wise – that is **gradient descent**. But the terrain is high-dimensional, non-convex, and you cannot see far. Steps too small crawl; steps too large overshoot ravines. Mini-batching, momentum, and adaptive steps are tricks for navigating this fog efficiently.

3.2 From GD to SGD

Full gradient descent uses all n examples:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t), \quad \nabla \mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\mathbf{w}_t) \tag{3.1}$$

For ImageNet ($n = 1.2$M), one step costs millions of forward passes – too slow.

Stochastic Gradient Descent (SGD) samples a mini-batch $\mathcal{B}$ of size B (32–256):

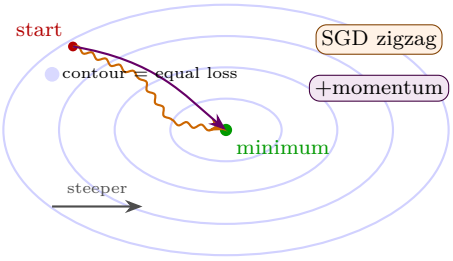
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}_t) \tag{3.2}$$

The mini-batch gradient is noisy but unbiased ($\mathbb{E}[\hat{\nabla}] = \nabla$). Noise helps escape sharp minima and generalises better.

Definition 3.1 (Momentum SGD)**.** Keep velocity $\mathbf{v}_t$:

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t + \hat{\nabla} \mathcal{L}(\mathbf{w}_t), \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \mathbf{v}_{t+1} \tag{3.3}$$

Typical $\mu = 0.9$. Momentum accelerates along consistent directions and damps oscillation – like a heavy ball rolling downhill.



3.3 Adam: Adaptive Moments

Different parameters need different step sizes (sparse features, ill-conditioning). **Adam** (Kingma & Ba, 2015) tracks per-parameter moments:

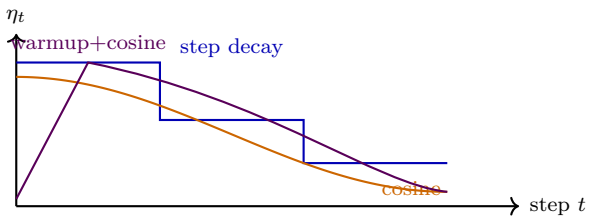
$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \\ \hat{\mathbf{m}}_t &= \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t) \\ \mathbf{w}_{t+1} &= \mathbf{w}_t - \eta \, \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon) \end{aligned} \tag{3.4}$$

Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$. $\mathbf{m}$ is momentum, $\mathbf{v}$ adapts scale – parameters with large gradients get smaller effective steps. Bias correction ($\hat{\cdot}$) fixes early-step under-estimate.

3.4 Learning Rate Schedules

A fixed η is rarely optimal. Common schedules:

- **Step decay:** $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/S \rfloor}$ (e.g., halve every 30 epochs).
- **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$.
- **Warmup:** linearly increase from 0 to $\eta_{\max}$ for first W steps, then decay – crucial for large batches and Transformers.



3.5 Batch Normalization

Internal covariate shift: as lower layers change, the distribution of inputs to upper layers shifts, forcing them to re-adapt. **BatchNorm** (Ioffe & Szegedy, 2015) normalizes each mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta \tag{3.5}$$

where $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$ are batch mean/variance, γ, β are learnable scale/shift, and at inference we use running averages. BatchNorm smooths the landscape, allows larger η, and has a mild regularization effect.

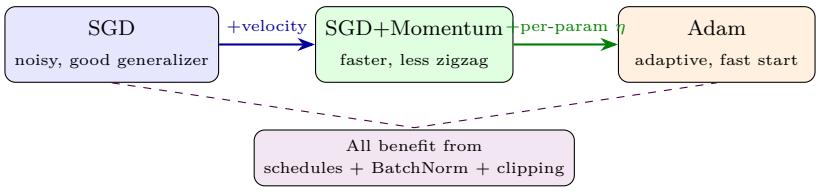
3.6 Worked Example: Tuning SGD vs Adam

Example 3.1 (PyTorch-like Training Loop).

```
import torch, math
# Toy: Rosenbrock loss, compare optimizers
def rosenbrock(p): return (1-p[0])**2 + 100*(p[1]-p[0]**2)**2

for name, Opt in [("SGD", torch.optim.SGD), ("Adam", torch.optim.Adam)]:
    p = torch.tensor([0.0, 0.0], requires_grad=True)
    opt = Opt([p], lr=0.01 if name=="Adam" else 0.001, momentum=0.9) if name=="SGD" else
        Opt([p], lr=0.01)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)
    for t in range(200):
        opt.zero_grad()
        loss = rosenbrock(p)
        loss.backward()
        # gradient clipping (common with RNNs/Transformers)
        torch.nn.utils.clip_grad_norm_([p], 1.0)
        opt.step(); sched.step()
```

```
    print(name, "final", p.detach().numpy(), "loss", loss.item())

# BatchNorm in a linear layer
import torch.nn as nn
layer = nn.Sequential(nn.Linear(64, 64), nn.BatchNorm1d(64), nn.ReLU())
x = torch.randn(32, 64)  # batch of 32
y = layer(x)  # normalized then scaled
print("BN output mean", y.mean().item(), "var", y.var().item())
```

3.7 Optimization Landscape



3.8 Gradient Noise and Generalization

Mini-batch noise is not just an artifact – it acts as implicit regularization. Small batches find flatter minima (Keskar et al., 2017) that generalize better, while large batches converge to sharper minima. The noise scale $g = \eta/B$ predicts this: keep g constant when scaling B and η. Second-order methods (Newton, L-BFGS) use curvature but scale poorly to millions of parameters; diagonal adaptivity (Adam) is the practical compromise.

3.9 Weight Initialization Revisited

Xavier and He initializations were derived to keep $\mathrm{Var}[a^{(l)}]$ constant across layers at initialization. For ReLU, He uses $W_{ij} \sim \mathcal{N}(0, 2/\mathrm{fan}_{in})$. Orthogonal init further helps RNNs. Poor init makes BatchNorm work harder and can trap optimization in symmetry.

3.10 Second-Order Intuition and Limitations

Newton's step $\mathbf{w} \leftarrow \mathbf{w} - H^{-1}\nabla\mathcal{L}$ uses the Hessian H to correct for curvature, jumping to the minimum of a quadratic approximation. But H is $d \times d$ ($d \sim 10^7$) – impossible to store or invert. Approximations (diagonal, K-FAC, L-BFGS) capture some curvature; Adam's $\mathbf{v}_t$ is a diagonal Hessian proxy. In practice, well-tuned SGD+momentum often generalizes slightly better than Adam on vision tasks, while Adam dominates for Transformers and noisy objectives.

3.11 Learning Rate Warmup and Clipping

Warmup avoids early divergence when gradients are large and Adam's second moment is uninitialized. Gradient clipping caps $\|\mathbf{g}\|$ to τ (e.g., 1.0): if $\|\mathbf{g}\| > \tau$, scale $\mathbf{g} \leftarrow \tau\mathbf{g}/\|\mathbf{g}\|$. Essential for RNNs/Transformers where backprop through many steps can spike. Together with cosine decay, warmup+clip is the standard recipe for stable large-scale training.

3.12 Common Pitfalls

Using Adam with default $\eta = 10^{-3}$ on a tiny dataset may overshoot; try 10^{-4} or SGD. Forgetting bias correction in custom Adam causes huge early steps. Not scheduling η wastes 30% of training time – a simple cosine decay is almost free. And never tune on test data: keep a held-out test set touched once, or your "improvements" are illusory.

3.13 Further Reading

Bottou et al. (2018) surveys optimization for ML; Kingma & Ba (2015) is the Adam paper; Loshchilov & Hutter (2019) fixes weight decay. For schedules, see Loshchilov & Hutter's `SGDR` and Smith's cyclical LR. Visualize loss landscapes with Li et al. (2018) filter-normalized plots – they explain why skip connections smooth the terrain.

3.14 Chapter Summary

Optimization navigates a non-convex landscape with noisy gradients. SGD + momentum accelerates; Adam adapts per-parameter steps via $\mathbf{m}, \mathbf{v}$. Schedules (cosine, warmup) and BatchNorm stabilize and accelerate. Gradient clipping cures explosions; small-batch noise regularizes. No optimizer dominates universally – tune η, B, λ jointly.

Key takeaways: (1) mini-batch variance $\propto 1/B$; (2) momentum damps zigzag; (3) Adam = momentum + RMS scaling + bias correction; (4) cosine + warmup is the modern schedule; (5) BatchNorm enables larger η and smoother loss.

Common pitfalls. Using the same batch statistics at test time, forgetting to shuffle data (correlated batches bias moments), and tuning on the test set are top-3 failures. Always maintain a held-out validation split and fix the random seed for reproducibility.

Example 3.2 (Adam step trace)**.** For $g_t = [0.1, -0.2]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, compute $m_1 = 0.09$, $v_1 = 0.004$, $\hat{m}_1 = 0.09/0.1 = 0.9$, $\hat{v}_1 = 0.004/0.001 = 4$, update $\Delta = -10^{-3} \cdot 0.9/(\sqrt{4} + 10^{-8}) \approx -4.5 \times 10^{-4}$. Small v keeps step moderate; without bias correction, early steps are damped by $1 - \beta$.

3.15 Exercises

Exercise 3.1. Derive the SGD update variance: if $\mathrm{Var}[\nabla \ell_i] = \sigma^2$, what is $\mathrm{Var}[\hat{\nabla}_{\mathcal{B}}]$ for batch size B? How does B trade noise vs compute?

Exercise 3.2. Adam with $\beta_1 = 0$ reduces to RMSProp-like scaling. Write the update and explain when momentum helps versus hurts (e.g., ravines vs saddle points).

Exercise 3.3. Implement cosine annealing from scratch and plot η_t for $T = 100$, $\eta_{\max} = 0.1$, $\eta_{\min} = 10^{-4}$. Add 10-step linear warmup and show the combined curve.

Exercise 3.4. Explain why BatchNorm uses different statistics at train vs test time. What goes wrong if you use batch statistics at inference with $B = 1$?

Exercise 3.5. Train a 3-layer MLP on MNIST with SGD ($\eta = 0.1$) and Adam ($\eta = 10^{-3}$). Compare loss curves. Now add BatchNorm – how does the optimal η shift?

Chapter 4

Regularization: Preventing Overfitting

A network with millions of weights can memorize your dataset – and fail on the next photo. Regularization teaches it to generalize.

From zero: no regularization background assumed. We start from memorization versus learning and build to weight decay, dropout, augmentation, and early stopping. Every formula is preceded by a picture; overfitting is defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) explain overfitting and the bias–variance tradeoff;

(L2) write the L_2-regularized loss and compare L_1/L_2 and elastic net;

(L3) explain how dropout, augmentation, and early stopping curb memorization;

(L4) diagnose under- and over-regularization from train/val curves.

4.1 Intuition: Memorization vs Learning

A 10M-parameter network can fit 50k CIFAR images perfectly, even with random labels – it merely memorizes. What we want is **generalization**: low error on unseen data. The gap between train and test error is **overfitting**. Regularization injects constraints or noise so the network cannot memorize quirks.

Think of fitting points with a polynomial: degree-15 fits noise wiggles; degree-3 captures the trend. The bias–variance tradeoff formalizes this: simple models underfit (high bias), complex models overfit (high variance), regularized models balance.

4.2 Weight Decay (L_2 Regularization)

Penalize large weights:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \|\mathbf{w}\|_2^2 = \mathcal{L}_{\text{data}} + \lambda \sum_i w_i^2 \tag{4.1}$$

Gradient step becomes

$$\mathbf{w} \leftarrow (1 - 2\eta\lambda)\mathbf{w} - \eta \nabla \mathcal{L}_{\text{data}} \tag{4.2}$$

so weights decay toward zero each step (hence "weight decay"). Large λ forces small weights $\rightarrow$ smoother functions $\rightarrow$ less overfitting. L_1 ($\lambda \sum |w_i|$) instead promotes sparsity.

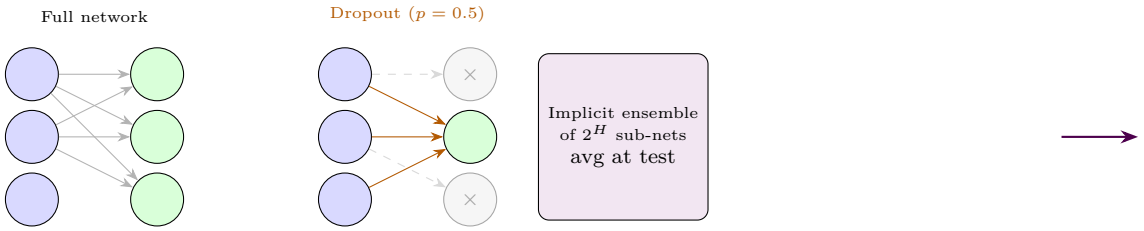
In AdamW (Loshchilov & Hutter, 2019), decay is decoupled: $\mathbf{w} \leftarrow \mathbf{w} - \eta(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon) + \lambda \mathbf{w})$, fixing Adam's improper L_2 interaction with adaptive scaling.

4.3 Dropout

During training, randomly zero each hidden unit with probability p (typically 0.2–0.5):

$$\tilde{\mathbf{a}}^{(l)} = \mathbf{m} \odot \mathbf{a}^{(l)}, \quad m_j \sim \text{Bernoulli}(1 - p) \tag{4.3}$$

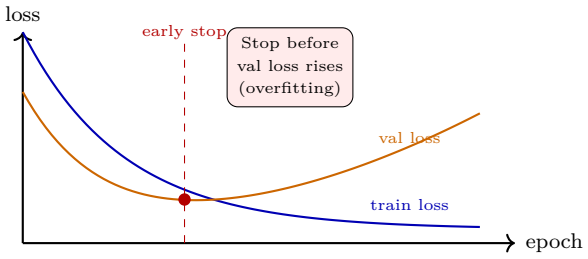
At test time, scale by $(1 - p)$ (or use inverted dropout: divide by $(1 - p)$ during train, no scaling at test). Dropout trains an implicit ensemble of 2^H sub-networks sharing weights; at inference we average them. It breaks co-adaptation: no neuron can rely on a specific partner being present.



4.4 Data Augmentation & Early Stopping

Augmentation expands data by label-preserving transforms: for images, random crop, flip, rotation, color jitter, Cutout/Mixup; for text, synonym replace, back-translation. More diverse data $\rightarrow$ less memorization.

Early stopping: monitor validation loss; stop when it rises for P epochs (patience). This is implicit L_2 control – fewer steps $\rightarrow$ weights stay nearer initialization (small norm). Combined with checkpointing the best model, it is the simplest regularizer that almost always helps.



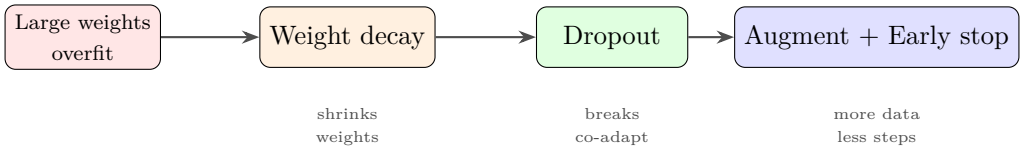
4.5 Putting It Together

Example 4.1 (PyTorch Regularization Cocktail).

```python
import torch.nn as nn
class RegularizedMLP(nn.Module):
    def __init__(self, p_drop=0.5, wd=1e-4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 512), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(512, 256), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(256, 10)
        )
        self.wd = wd
    def forward(self, x): return self.net(x.view(x.size(0),-1))

model = RegularizedMLP(p_drop=0.3)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)  # decoupled wd
aug = nn.Sequential()  # placeholder: use torchvision.transforms
# Training loop with early stopping
best_val, patience, wait = float('inf'), 5, 0
for epoch in range(100):
    model.train()
    for x,y in train_loader:  # x augmented: RandomCrop, HorizontalFlip
        opt.zero_grad()
        loss = nn.CrossEntropyLoss()(model(x), y)
        loss.backward(); opt.step()
    # validation
    model.eval()
    val_loss = sum(nn.CrossEntropyLoss()(model(x),y).item() for x,y in val_loader)/len(
        val_loader)
    if val_loss < best_val: best_val=val_loss; wait=0; torch.save(model.state_dict(),"best
        .pt")
    else: wait+=1
    if wait>=patience: print(f"Early stop at {epoch}"); break
model.load_state_dict(torch.load("best.pt"))
```

A complementary technique is **label smoothing**: replace one-hot y_k with $(1-\epsilon)y_k + \epsilon/K$, discouraging overconfident logits.

4.6 Regularization Spectrum



4.7 Bias–Variance Tradeoff Formalized

For squared loss, expected test error decomposes: $\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Var} + \sigma^2_{\text{noise}}$. Capacity ↑ lowers bias but raises variance. Regularization traces a U-curve: too little → overfit (high var), too much → underfit (high bias). Validation curves (train vs val loss versus λ or p) locate the sweet spot. Cross-validation refines this when data are scarce.

4.8 Weight Decay vs L_1 and Elastic Net

L_2 shrinks weights smoothly; L_1 ($\lambda \sum |w_i|$) yields sparsity via soft-thresholding – useful for feature selection. Elastic net mixes both: $\lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$. In deep nets, L_2 is dominant because we rarely want exact zeros in dense layers, but L_1 appears in pruning and compressed sensing. Decoupled AdamW correctly separates decay from gradient-based adaptivity, fixing the original Adam+L_2 bug.

4.9 Dropout Variants and Analysis

Standard dropout drops units; **DropConnect** drops weights, **Spatial Dropout** drops whole feature maps (better for CNNs where neighbours correlate), **Cutout/RandomErasing** drops image patches. At test time, Monte Carlo dropout (keep dropout on, average many forwards) estimates uncertainty – a cheap Bayesian approximation (Gal & Ghahramani, 2016). Inverted dropout's scaling by $1/(1-p)$ during training keeps expected activation magnitude constant, so no test-time rescaling is needed.

4.10 Augmentation Zoo and Mixup

Beyond crop/flip, modern augmentation includes: color jitter, Gaussian blur, Mixup ($\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$, $\tilde{y}$ mixed similarly – encourages linear behaviour between examples), CutMix (paste patches), and AutoAugment (learned policies). For NLP, back- translation and EDA (random insert/delete/swap) help. Strong augmentation can replace substantial labeled data – a key reason ImageNet models train on "infinite" augmented streams rather than 1.2M fixed images.

4.11 Common Pitfalls

Applying dropout at test time without scaling ruins predictions – use `model.eval()` in PyTorch to disable it. Setting λ too high underfits; too low does nothing – sweep λ on a log scale and watch the train–val gap. Augmentation that is too strong (e.g., 90-degree rotation on digit 6→9) changes labels and hurts. Validate augmentation by visual inspection.

4.12 Further Reading

Srivastava et al. (2014) on dropout; Zhang et al. (2017) "Understanding deep learning requires rethinking generalization" shows memorization power. For augmentation, see Cubuk et al. AutoAugment and Zhang et al. Mixup. Early stopping theory is in Yao et al. (2007) – L_2 equivalence.

4.13 Chapter Summary

Capacity without constraint memorizes. L_2 shrinks weights; dropout trains an ensemble of sub-nets; augmentation enlarges data; early stopping limits steps. Together they close the train–val gap. AdamW decouples decay correctly; Mixup and label smoothing further curb overconfidence. Regularization is not optional at scale – it is what makes generalization possible.

Key takeaways: (1) $\mathcal{L} + \lambda\|\mathbf{w}\|^2$ decays weights; (2) dropout $p = 0.2$–0.5 breaks co-adaptation; (3) augmentation is free data; (4) early stopping $\approx L_2$; (5) AdamW > Adam+L_2 for adaptive methods.

Debugging regularization. If train loss is high, regularization is too strong—reduce λ or p. If val loss diverges early, increase augmentation. Plot weight histograms: L_2 keeps them Gaussian, dropout keeps them sparse.

Example 4.2 (Dropout expectation)**.** For input $x = 2$ and mask $m \sim$ Bernoulli(0.5), training output $y = m \cdot x/0.5$ has $\mathbb{E}[y] = 2$. Inverted dropout scales at train time, so test time needs no scaling. This preserves $\mathbb{E}[y_{\text{train}}] = \mathbb{E}[y_{\text{test}}]$.

Remark 4.1 (Label smoothing). Replacing hard label 1 with 0.9 and 0 with $0.1/(K-1)$ prevents overconfident logits, reducing $\|\mathbf{z}\|_2$ and improving calibration.

4.14 Exercises

Exercise 4.1. Show that L_2-regularized linear regression has closed form $\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$. What happens as $\lambda \to \infty$?

Exercise 4.2. Dropout with $p = 0.5$ on a layer of 1000 units: how many distinct sub-networks? Why is the test-time scaling by 0.5 needed for expectation matching?

Exercise 4.3. Implement data augmentation for CIFAR-10: random crop + horizontal flip. Train a small CNN with and without augmentation; report train/test gap.

Exercise 4.4. Early stopping as regularization: sketch why stopping after T SGD steps is similar to L_2 with $\lambda \propto 1/T$. Hint: gradient flow path length.

Exercise 4.5. Compare Adam vs AdamW on a 4-layer MLP with $\lambda = 10^{-2}$. Why does AdamW generalize better? Inspect effective weight-decay magnitude.

Chapter 5

Convolutional Neural Networks

Images are not flat vectors – they have locality, translation, and scale. Convolutions exploit all three.

From zero: no vision background assumed. We start from why MLPs fail on images and build to convolution, pooling, and residual blocks. Every formula is preceded by a picture; kernels and feature maps are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) explain locality, sharing, and translation equivariance of convolutions;

(L2) write 2D convolution and count parameters and receptive fields;

(L3) describe VGG-to-ResNet blocks and why residuals train deeper;

(L4) build a small CNN from scratch and avoid common training pitfalls.

5.1 Intuition: Why Not an MLP for Images?

A $224 \times 224 \times 3$ image flattened is 150k inputs. An MLP hidden layer with 1000 units needs 150M weights – huge and blind to structure. Nearby pixels correlate (edges, textures); an object shifted by 10 pixels is still the same object. An MLP must relearn detectors at every position. **Convolution** shares weights across space: one small filter slides everywhere, detecting the same pattern regardless of location. This gives **translation equivariance** and drastic parameter savings.

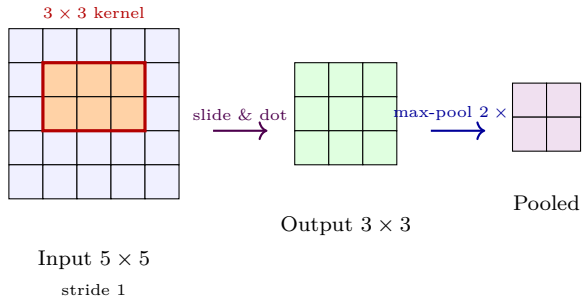
5.2 Convolution and Pooling

Definition 5.1 (2D Convolution (cross-correlation))**.** Given input $I \in \mathbb{R}^{H\times W}$ and kernel $K \in \mathbb{R}^{k\times k}$,

$$(I * K)_{i,j} = \sum_{u=0}^{k-1}\sum_{v=0}^{k-1} I_{i+u,\,j+v}\, K_{u,v} \tag{5.1}$$

For C_{in} input channels and C_{out} filters, $K \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}\times k\times k}$. Stride s and padding p control output size: $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$.

Pooling downsamples: **max-pool** takes the max in each 2×2 window (stride 2 halves resolution, keeps strongest activation, adds small translation invariance); average-pool smooths.



A CNN stacks: `Conv` → `BatchNorm` → `ReLU` → `Pool`, repeating. Early layers learn Gabor-like edge detectors; deeper layers compose them into textures, parts, objects – a learned hierarchy.

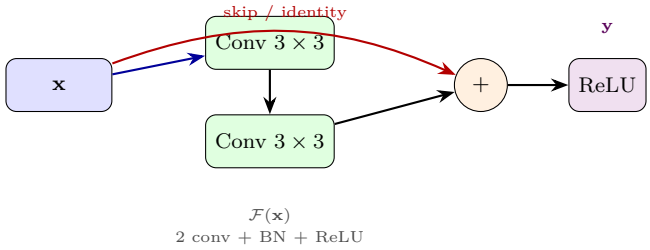
5.3 Classic Architectures to ResNet

- **LeNet-5** (1998): 2 conv+pool + 3 FC – reads digits.
- **AlexNet** (2012): 5 conv + 3 FC, ReLU, dropout, GPU – wins ImageNet, sparks deep learning boom.
- **VGG** (2014): only 3×3 conv, deeper (16–19 layers) – simplicity.
- **GoogLeNet/Inception**: parallel multi-scale filters.
- **ResNet** (2015): solves degradation (deeper nets got *worse*) via skip connections.

Definition 5.2 (Residual Block)**.** Instead of learning $\mathcal{H}(\mathbf{x})$, learn residual $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{5.2}$$

If identity is optimal, the net can push $\mathcal{F} \to 0$ (easy) rather than fit identity through non-linear layers (hard). Gradient flows directly through the skip, curing vanishing gradients.



ResNet-18/50/152 stack such blocks, reaching 100+ layers with *lower* training error than 20-layer nets – depth finally helps. BatchNorm inside each block stabilizes.

5.4 Worked Example: CNN from Scratch

Example 5.1 (PyTorch CNN for CIFAR-10).

```
import torch.nn as nn
class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3,32,3,padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),  # 32 -> 16
            nn.Conv2d(32,64,3,padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),  # 16 -> 8
            nn.Conv2d(64,128,3,padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1)  # -> (128,1,1)
        )
        self.classifier = nn.Linear(128, 10)
    def forward(self, x):
        x = self.features(x).flatten(1)
        return self.classifier(x)

# Residual block
class ResBlock(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.conv1 = nn.Conv2d(c,c,3,padding=1,bias=False)
        self.bn1 = nn.BatchNorm2d(c)
        self.conv2 = nn.Conv2d(c,c,3,padding=1,bias=False)
        self.bn2 = nn.BatchNorm2d(c)
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return torch.relu(out + x)  # skip connection

# Count params: TinyCNN ~ 100k, ResNet-18 ~ 11M
model = TinyCNN()
print(f"Params: {sum(p.numel() for p in model.parameters())}")
```

A pure NumPy convolution inner loop is also instructive:

```python
import numpy as np
def conv2d_numpy(img, kernel):
    H,W = img.shape; kH,kW = kernel.shape
    out = np.zeros((H-kH+1, W-kW+1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i,j] = np.sum(img[i:i+kH, j:j+kW] * kernel)
    return out

# Sobel edge detector
sobel_x = np.array([[-1,0,1],[-2,0,2],[-1,0,1]], dtype=float)
edges = conv2d_numpy(np.random.randn(28,28), sobel_x)
```

5.5 Vision Beyond Classification

CNN features transfer: the same backbone serves detection (Faster R-CNN, YOLO – bounding boxes), segmentation (U-Net – per-pixel masks), and self-supervised pretraining. Modern vision mixes CNNs with Transformers (ViT), but convolution remains the efficient inductive bias for grid data.

5.6 Receptive Field and Parameter Counting

A 3×3 conv has $9C_{in}C_{out}$ weights – independent of image size. Stacking two 3×3 layers yields a 5×5 receptive field with fewer parameters ($2\cdot9$ vs 25 per channel) and an extra non-linearity. After L layers of stride 1, receptive field $= 1+L(k-1)$. Stride or pooling exponentially expands it. Dilated (atrous) conv inserts gaps to grow field without downsampling – vital for segmentation.

5.7 Translation Equivariance Proof Sketch

Let T_Δ shift input by Δ. Then $(T_\Delta I)*K = T_\Delta(I*K)$ because summation indices shift uniformly – convolution commutes with translation. Pooling is approximately invariant: $\text{Pool}(T_\Delta I)\approx\text{Pool}(I)$ for $|\Delta|<$ pool window, giving robustness to small jitters. Fully-connected layers lack this property, hence CNNs need far less data for vision.

5.8 Architectures in Detail: VGG to ResNet to EfficientNet

VGG showed depth matters; GoogLeNet showed multi-scale matters. ResNet made depth *trainable*: 152 layers beat 34, and 1001-layer ResNets were trained. Follow-ups: **DenseNet** concatenates all prior feature maps; **ResNeXt** adds grouped conv; **EfficientNet** compound-scales width/depth/resolution with neural architecture search. All retain convolution as the workhorse; recent hybrids replace late conv stages with Transformer blocks (CoAtNet).

5.9 Efficient CNN Training Tricks

Use He init, BatchNorm after conv (before ReLU), weight decay 10^{-4}, SGD with momentum 0.9 and cosine decay, label smoothing 0.1, and Mixup/CutMix. Progressive resizing (start 128^2, end 224^2) speeds training. Depthwise separable conv (MobileNet) factorizes $k \times k \times C_{in} \times C_{out}$ into $k \times k \times C_{in}$ (spatial) + $1 \times 1 \times C_{in} \times C_{out}$ (pointwise) for 8–9× savings on mobile.

5.10 Common Pitfalls

Forgetting padding causes spatial shrinkage and border artifacts – use `padding=1` for 3×3 stride 1 to preserve size. Not normalizing inputs (ImageNet mean/std) slows convergence by an order of magnitude. Using large 7×7 kernels where two 3×3 suffice wastes compute. And placing dropout before BatchNorm is unstable – order matters: Conv → BN → ReLU → Dropout (or use DropBlock for conv).

5.11 Further Reading

LeCun et al. (1998) LeNet; Krizhevsky et al. (2012) AlexNet; Simonyan & Zisserman (2014) VGG; He et al. (2016) ResNet. For modern practice, see Howard & Gugger, *Deep Learning for Coders*, and the `timm` library – every architecture in one repo, ideal for ablations.

5.12 Chapter Summary

Convolutions share weights spatially: $k \times k$ filters slide, giving translation equivariance and $O(k^2 C_{in} C_{out})$ params. Pooling downsamples and adds invariance. Stacking yields hierarchical features; residuals let depth scale to 100+ layers. ResNet's $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ is the trick that unlocked depth. The same conv backbone powers detection, segmentation, and beyond.

Key takeaways: (1) conv = local + shared + equivariant; (2) pooling ≈ invariant; (3) receptive field grows linearly then exponentially; (4) ResNet skip cures degradation; (5) depthwise separable conv saves 8× for mobile.

5.13 Exercises

Exercise 5.1. Compute output size for input $32 \times 32 \times 3$, kernel 5×5, stride 1, padding 2, 16 filters. How many parameters (with bias)? Compare to an MLP with same output dimension.

Exercise 5.2. Prove convolution is translation equivariant: shifting input then convolving equals convolving then shifting. Why is pooling *invariant* (approximately)?

Exercise 5.3. Implement NumPy max-pool 2×2 stride 2. Test on a 4×4 matrix and verify gradients route only to max positions.

Exercise 5.4. Build a ResBlock and show that if weights are zero, it computes identity. Explain why this makes optimization easier than a plain stack.

Exercise 5.5. Train TinyCNN on CIFAR-10 for 5 epochs. Replace AdaptiveAvgPool with Flatten+Linear and compare parameter counts and accuracy. When does global average pooling help?

Chapter 6

Recurrent Networks & Sequence Modeling

Language, speech, and time series unfold step by step – networks must remember.

From zero: no sequence background assumed. We start from why fixed windows fail on language and build from Elman RNNs to LSTM/GRU gating. Every formula is preceded by a picture; hidden states are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) write the Elman RNN recurrence and its unrolling through time;

(L2) explain vanishing and exploding gradients in vanilla RNNs;

(L3) write LSTM/GRU gate equations and explain what each gate does;

(L4) compare bidirectional/stacked RNNs and the bridge to attention.

6.1 Intuition: Memory Through Loops

An MLP sees a fixed window; language does not fit. "The cat that chased the dogs *was*..." – agreement depends on "cat" six words back. We need a network with **memory**: a hidden state that carries information forward. A **Recurrent Neural Network (RNN)** does exactly this – it reuses the same weights at each time step, updating a state $\mathbf{h}_t$ from input $\mathbf{x}_t$ and previous state $\mathbf{h}_{t-1}$.

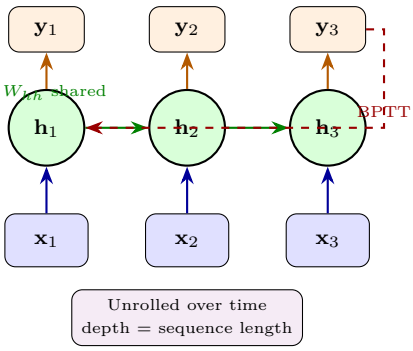
Unrolling the loop in time reveals a deep network whose depth is the sequence length – with shared weights and a gradient that must flow back through time.

6.2 The Vanilla RNN

Definition 6.1 (Elman RNN)**.** For sequence $\mathbf{x}_{1:T}$,

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad \mathbf{h}_0 = \mathbf{0} \tag{6.1}$$

Loss sums over time: $\mathcal{L} = \sum_t \ell(\mathbf{y}_t, \mathbf{y}_t^*)$.



Backpropagation Through Time (BPTT): unroll, then backprop. Gradient involves repeated multiplication by W_{hh}:

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{t=2}^{T} W_{hh}^\top \operatorname{diag}(\tanh'(\cdot)) \tag{6.2}$$

If spectral radius < 1, gradient vanishes exponentially; if > 1, it explodes. This is why vanilla RNNs cannot learn dependencies beyond $\sim$10 steps.

6.3 LSTM and GRU: Gated Memory

Gates – learnable sigmoidal switches in $[0, 1]$ – protect the gradient by creating an additive "conveyor belt".

Definition 6.2 (LSTM, Hochreiter & Schmidhuber 1997)**.**

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \qquad \text{forget gate} \tag{6.3}$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \qquad \text{input gate} \tag{6.4}$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \qquad \text{candidate} \tag{6.5}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \qquad \text{cell state (additive!)} \tag{6.6}$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \qquad \text{output gate} \tag{6.7}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{6.8}$$

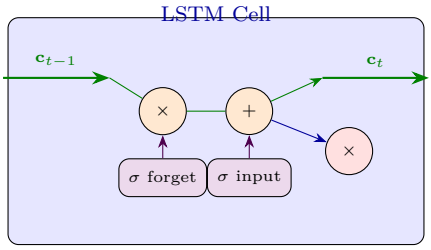
Key: $\mathbf{c}_t$ updates *additively*; gradient $\partial \mathbf{c}_T / \partial \mathbf{c}_t = \prod \mathbf{f}_k$ stays near 1 if forget gates stay open.

Definition 6.3 (GRU, Cho et al. 2014)**.** Simpler, two gates:

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \tag{6.9}$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{6.10}$$

Often matches LSTM with fewer parameters.



6.4 Attention Intuition (Bridge to Transformers)

RNNs compress all history into $\mathbf{h}_t$ – a bottleneck. **Attention** lets the decoder peek at all encoder states: for each output step, compute weights $\alpha_{t,i} = \text{softmax}(\text{score}(\mathbf{s}_{t-1}, \mathbf{h}_i))$ and context $\mathbf{c}_t = \sum_i \alpha_{t,i}\mathbf{h}_i$. The network *learns where to look*. This idea, scaled up and stripped of recurrence, becomes the Transformer.

6.5 Worked Example

Example 6.1 (Character-Level RNN in PyTorch).

```python
import torch, torch.nn as nn
class CharRNN(nn.Module):
    def __init__(self, vocab=65, hidden=128):
        super().__init__()
        self.emb = nn.Embedding(vocab, 64)
        self.rnn = nn.LSTM(64, hidden, batch_first=True)  # or nn.GRU
        self.fc  = nn.Linear(hidden, vocab)
    def forward(self, x, h=None):
        e = self.emb(x)           # (B,T,64)
        out, h = self.rnn(e, h)  # out: (B,T,hidden)
        return self.fc(out), h   # logits per step

# Training: teacher forcing
model = CharRNN(); opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for x,y in loader:  # x:(B,T) y:(B,T) next-char targets
    opt.zero_grad()
    logits,_ = model(x)
    loss = nn.CrossEntropyLoss()(logits.view(-1,65), y.view(-1))
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # cure explosion
    opt.step()

# Sampling
model.eval()
x = torch.tensor([[ord('T')%65]])
with torch.no_grad():
    for _ in range(50):
        logits,h = model(x[:,-1:])
        nxt = torch.multinomial(torch.softmax(logits[:,-1],-1),1)
```

```
        x = torch.cat([x,nxt],1)
print(''.join(chr(c.item()) for c in x[0]))
```

Bidirectional RNNs run forward and backward, concatenating states for tasks like NER where future context matters. Stacking 2–4 layers deepens representation.

6.6 Why Vanilla RNNs Fail: Vanishing/Exploding Gradients

Consider $\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + \dots)$. Then $\partial \mathbf{h}_T / \partial \mathbf{h}_t = \prod_{k=t+1}^{T} D_k W_{hh}$ where $D_k = \mathrm{diag}(\tanh'(\cdot))$ has entries in $(0, 1]$. Norm behaves as $\|W_{hh}\|^{T-t}$. If $\|W_{hh}\| < 1$, product $\to 0$ exponentially; if > 1, $\to \infty$. Gradient clipping handles explosion, but vanishing requires architecture – gates. Orthogonal init of W_{hh} (eigenvalues ≈ 1) helps but is insufficient alone for long horizons.

6.7 LSTM Gates in Action

Walk through a memory write/read: to remember a fact at $t = 5$ for use at $t = 50$, the forget gate must stay ≈ 1 for 45 steps ($\mathbf{b}_f$ initialized to 1 helps), input gate opens at $t = 5$ to write $\tilde{\mathbf{c}}_5$, cell holds $\mathbf{c}_t$ additively, output gate opens at $t = 50$ to expose it. Peephole connections (optional) let gates see $\mathbf{c}$. GRU merges cell and hidden, using update gate $\mathbf{z}_t$ to interpolate old vs new – fewer parameters, similar performance, faster.

6.8 Bidirectional and Stacked RNNs

A **Bi-RNN** runs forward $\overrightarrow{\mathbf{h}}_t$ and backward $\overleftarrow{\mathbf{h}}_t$, concatenating $[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ for tasks where future context helps (NER, speech). **Stacked** (deep) RNNs feed $\mathbf{h}_t^{(l)}$ as input to layer $l+1$, learning hierarchical timescales. Attention over encoder states (Bahdanau, Luong) then lets decoders focus: $\alpha_{t,i} \propto \exp(\mathbf{s}_{t-1}^{\top} W \mathbf{h}_i)$, $\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$, finally $\tilde{\mathbf{s}}_t = \tanh(W_c[\mathbf{c}_t; \mathbf{s}_t])$ before predicting y_t.

6.9 RNNs Today and the Path to Transformers

LSTMs dominated 2014–2017 for translation, speech, and language modeling. But sequential dependence prevents parallelization ($O(T)$ steps) and path length between positions is $O(T)$. Attention shortens it to $O(1)$ and parallelizes to $O(1)$ steps with $O(T^2)$ work – the Transformer keeps the gating insight (additive paths, normalization) but removes recurrence entirely.

6.10 Generative Models: Autoencoders, VAEs, GANs, Diffusion

Our CharRNN already *generates*: given history it predicts the next character, and sampling turns those predictions into new text. Modern generative models generalize this idea – learn the data distribution $p(\mathbf{x})$ itself, then sample brand-new $\mathbf{x}$ from it. Three landmarks cover the design space.

Autoencoder to VAE. An **autoencoder** compresses $\mathbf{x}$ through a bottleneck $\mathbf{z} = f_{\text{enc}}(\mathbf{x})$ and reconstructs $\hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z})$, trained to minimize $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. A **variational autoencoder** (Kingma–Welling, 2013) makes $\mathbf{z}$ a distribution $q(\mathbf{z} \mid \mathbf{x})$ with prior $p(\mathbf{z}) = \mathcal{N}(0, I)$, so we can sample $\mathbf{z} \sim p(\mathbf{z})$ and decode novel $\mathbf{x}$. Training maximizes the ELBO one-liner:

$$\log p(\mathbf{x}) \geq \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - \mathrm{KL}(q(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})). \tag{6.11}$$

The first term pulls reconstructions toward realism; the KL term keeps the latent space dense and samplable – without it, gaps in $\mathbf{z}$-space decode to garbage.

GANs. A **GAN** (Goodfellow et al., 2014) pits a generator $G(\mathbf{z})$ against a discriminator $D(\mathbf{x})$ in a minimax game: $\min_G \max_D \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$. D learns to spot fakes, G learns to fool D; at equilibrium G draws from p_{data}. Samples are sharp, but training can collapse to a few modes and offers no likelihood.

Diffusion. A **diffusion model** (Ho et al., 2020) destroys structure by adding Gaussian noise over T steps, then learns the reverse: a denoiser $\epsilon_\theta(\mathbf{x}_t, t)$ predicting the noise present at each step. Sampling starts from pure noise $\mathbf{x}_T \sim \mathcal{N}(0, I)$ and iteratively denoises down to $\mathbf{x}_0$. Slow (many steps) but stable, with the best image quality to date. Our CharRNN sampler is its autoregressive cousin – both turn a seed (or noise) into data one refinement at a time.

6.11 Common Pitfalls

Feeding the whole sequence without truncation causes $O(T)$ memory blowup and slow BPTT – use truncated BPTT (backprop every k steps, carry state). Not detaching hidden states between batches leaks gradient across unrelated sequences. Teacher forcing during training vs autoregressive sampling at test causes exposure bias – scheduled sampling or professor forcing mitigates. Always clip gradients; without clipping, one bad batch can NaN the run.

6.12 Further Reading

Hochreiter & Schmidhuber (1997) LSTM; Cho et al. (2014) GRU; Bahdanau et al. (2015) attention. Graves (2013) applies RNNs to speech; Karpathy's "The Unreasonable Effectiveness of RNNs" is the classic blog. For BPTT theory, see Pascanu et al. (2013) on vanishing/exploding.

6.13 Chapter Summary

RNNs share weights over time: $\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$. BPTT unrolls and backprops through the product of Jacobians – vanishing/exploding follows. LSTM/GRU gates create additive memory ($\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$) preserving gradient. Attention fixes the bottleneck by peeking at all states, foreshadowing Transformers.

Key takeaways: (1) recurrence = memory via loop; (2) BPTT = backprop through time; (3) gates = learnable switches; (4) $\mathbf{f} \approx 1$ preserves long memory; (5) attention > compression.

Gradient sanity check. Before training, run gradient checking with $\epsilon = 10^{-5}$ on a 3-step unroll. If relative error $> 10^{-3}$, check gate biases (forget bias $b_f = 1$ helps) and clipping threshold.

Example 6.2 (BPTT product)**.** For $h_t = \tanh(Wh_{t-1} + Ux_t)$, $\partial h_3/\partial h_1 = \prod_{k=2}^{3} \mathrm{diag}(1 - \tanh^2(\cdot))W$. Spectral radius $\rho(W) > 1$ explodes, $\rho(W) < 1$ vanishes. Gating replaces product with sum via c_t.

6.14 Exercises

Exercise 6.1. Unroll a vanilla RNN for $T = 3$ and derive $\partial \mathcal{L}_3/\partial W_{hh}$ explicitly. Identify the product that vanishes/explodes.

Exercise 6.2. Explain in words how LSTM's forget gate $\mathbf{f}_t \approx 1$ preserves gradient over 100 steps. What initialization of $\mathbf{b}_f$ encourages this?

Exercise 6.3. Implement a NumPy forward pass for one LSTM step. Count parameters for $d_x = 64$, $d_h = 128$ and compare to GRU and vanilla RNN.

Exercise 6.4. Train CharRNN on Tiny Shakespeare (10k chars) with RNN vs LSTM (same hidden size). Plot loss and report which captures longer dependencies – test by checking if it closes a quote 30 chars later.

Exercise 6.5. For seq2seq translation, why does attention help with long sentences? Sketch the alignment matrix for "The cat sat" $\rightarrow$ "Le chat s'est assis".

Chapter 7

Transformers & Attention

If attention is all you need, why recur at all? A model that looks everywhere at once.

From zero: no attention background assumed. We start from the cost of recurrence and build to self-attention, multi-head blocks, and the BERT/GPT paradigms. Every formula is preceded by a picture; queries, keys, and values are defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) write scaled dot-product self-attention and multi-head attention;

(L2) describe a Transformer block (attention, residual, normalization, FFN);

(L3) contrast BERT (encoder) and GPT (decoder) training objectives;

(L4) explain positional encodings and $O(n^2)$ scaling with efficient fixes.

7.1 Intuition: From Recurrence to Attention

RNNs are sequential – step t waits for $t-1$. This blocks parallelization and still struggles with very long range. Transformers (Vaswani et al., 2017) ask: what if each position directly attends to *all* positions? No recurrence, no convolution – just weighted averaging where weights are learned from data. The result is $O(1)$ path length between any two positions, perfect parallelism, and scaling to billions of parameters.

The core is **self-attention**: each token creates a query, key, value; queries match keys to decide how much value to pull.

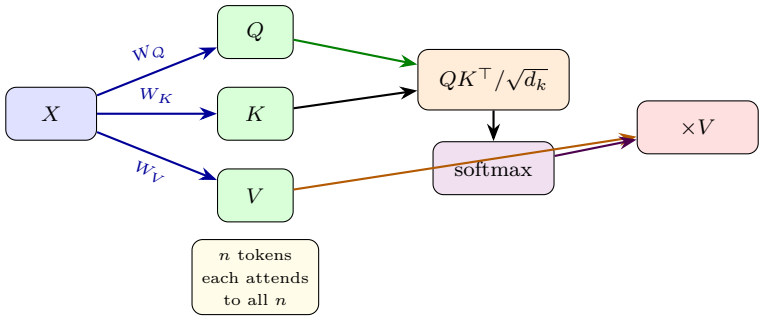
7.2 Self-Attention Formally

Given input $X \in \mathbb{R}^{n \times d}$ (n tokens, d dims), learn $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ (often $d_k = d/h$ for h heads):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{7.1}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{7.2}$$

$QK^\top \in \mathbb{R}^{n \times n}$ holds pairwise scores; $\sqrt{d_k}$ scaling prevents softmax saturation; softmax rows sum to 1, giving convex combinations of V. Intuition: query asks "what am I looking for?", key says "what do I contain?", value is "what I contribute".



Multi-Head Attention runs h independent attentions in parallel (each with $d_k = d/h$), concatenates, then projects: $\text{MH}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$. Different heads learn different relations (syntax, coreference, position).

Positional Encoding: attention is permutation-invariant, so we inject order. Original sinusoid:

$$\text{PE}_{(pos,2i)} = \sin(pos/10000^{2i/d}), \quad \text{PE}_{(pos,2i+1)} = \cos(pos/10000^{2i/d}) \tag{7.3}$$

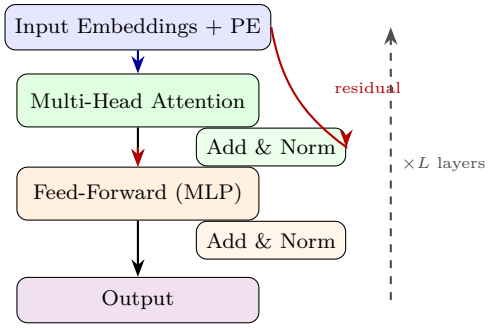
or learned embeddings. Added to input: $X \leftarrow X + \text{PE}$.

7.3 The Transformer Block

Each layer (repeated $L = 6$–96 times):

$$\begin{aligned} \mathbf{Z} &= \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttn}(\mathbf{X})) \\ \mathbf{Y} &= \text{LayerNorm}(\mathbf{Z} + \text{FFN}(\mathbf{Z})), \quad \text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2 \end{aligned} \tag{7.4}$$

Residual + LayerNorm stabilize deep stacking. Masked attention in the decoder prevents looking at future tokens (causal).



7.4 BERT and GPT: Two Paradigms

BERT (Devlin et al., 2018): encoder-only, bidirectional, pre-trained by *masked LM* (mask 15% tokens, predict them) + next-sentence. Fine-tune on downstream tasks. Sees left and right context – great for understanding.

GPT (Radford et al., 2018–2023): decoder-only, causal, pre-trained by *next-token prediction*: $p(x_1, \ldots, x_n) = \prod_t p(x_t \mid x_{<t})$. Generate by sampling autoregressively. Scales predictably – larger models + more data $\rightarrow$ emergent abilities (few-shot, chain-of-thought). ChatGPT, GPT-4 are large GPTs with RLHF.

Vision Transformer (ViT) splits images into 16×16 patches and treats them as tokens – same architecture, new modality.

7.5 Worked Example

Example 7.1 (Self-Attention in NumPy and PyTorch).

```
import numpy as np
def self_attention_numpy(X, Wq, Wk, Wv):
    Q, K, V = X@Wq, X@Wk, X@Wv          # (n, d_k)
    scores = Q @ K.T / np.sqrt(K.shape[1])
    weights = np.exp(scores)
    weights /= weights.sum(axis=-1, keepdims=True)  # softmax rows
    return weights @ V, weights

# Tiny demo: 4 tokens, d=8, d_k=8
np.random.seed(0)
X = np.random.randn(4,8)
Wq,Wk,Wv = [np.random.randn(8,8)*0.3 for _ in range(3)]
out, w = self_attention_numpy(X, Wq, Wk, Wv)
print("Attention weights row 0:", w[0].round(2))  # sums to 1
print("Output shape", out.shape)

# PyTorch multi-head (single layer)
import torch, torch.nn as nn
layer = nn.TransformerEncoderLayer(d_model=64, nhead=4, dim_feedforward=256, batch_first=
    True)
enc = nn.TransformerEncoder(layer, num_layers=2)
x = torch.randn(2, 10, 64)  # (batch, seq, dim)
mask = torch.triu(torch.ones(10,10), diagonal=1).bool()  # causal mask
y = enc(x, mask=mask)
print("Encoded", y.shape)  # (2,10,64)
```

```
# GPT-style generation: next-token loop uses cached K,V (KV-cache) for speed
```

7.6 Multi-Head Attention in Depth

With h heads, each head i has $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d_k}$ where $d_k = d/h$. Head output $H_i = \text{Attn}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}) \in \mathbb{R}^{n \times d_k}$. Concatenate and project: $\text{MH}(X) = [H_1; \ldots; H_h]W_O$, $W_O \in \mathbb{R}^{d \times d}$. Different heads specialize: one tracks syntax, another coreference, another position. Visualizing $\text{softmax}(QK^\top/\sqrt{d_k})$ reveals interpretable alignments – e.g., "it" attending to its antecedent.

7.7 Positional Encodings Compared

Sinusoidal PE gives extrapolation to longer sequences (periodic, no learned params). Learned PE is more flexible but fails beyond training length. Relative PE (Shaw et al.) and RoPE (Su et al., 2021) encode $pos_i - pos_j$ directly and are now standard in LLMs for long-context (4k–128k tokens). ALiBi adds a linear bias to attention scores proportional to distance – simple and effective for extrapolation. The choice impacts length generalization dramatically.

7.8 Training at Scale: Normalization and Stability

Pre-norm vs post-norm: original Transformer used post-norm (Norm after residual), but deep stacks (> 24 layers) train more stably with pre-norm (Norm before attention/FFN, residual is clean path). Variants: RMSNorm, LayerNorm without bias, and DeepNorm for 1000-layer Transformers. Mixed-precision (FP16/BF16) with loss scaling, gradient accumulation, and ZeRO sharding enable trillion- parameter training across thousands of GPUs. Without these, large Transformers diverge within hundreds of steps.

7.9 From Language to Vision and Beyond

ViT (Dosovitskiy et al., 2020) reshapes 224×224 into $14 \times 14 = 196$ patches of 16×16, linearly embeds each, adds PE, and feeds a standard Transformer – beating CNNs when pre-trained on large data (JFT-300M). Swin uses shifted windows for efficiency. Perceiver, DETR, and diffusion Transformers extend attention to detection, generation, and multimodal fusion – attention is now the universal primitive, not just for language.

7.10 Efficiency and Sparse Attention

Full attention is $O(n^2)$ – prohibitive for $n = 100$k. Sparse variants: Longformer (sliding window + global), BigBird, FlashAttention (IO-aware tiling that is exact but 2–4× faster), and linear attention ($O(n)$ via kernel trick). KV-cache during autoregressive generation stores past K, V to avoid recomputation, cutting per-token cost from $O(n^2)$ to $O(n)$.

7.11 Common Pitfalls

Forgetting the causal mask in decoder training lets the model cheat by seeing the future – validation perplexity collapses but generation is gibberish. Not scaling by $\sqrt{d_k}$ saturates softmax and gradients vanish. Using post-norm with > 24 layers often diverges; switch to pre-norm. And positional encodings must match max sequence at inference – extrapolating learned PE without interpolation fails silently.

7.12 Further Reading

Vaswani et al. (2017) "Attention Is All You Need"; Devlin et al. (2019) BERT; Radford et al. (2018, 2019) GPT/GPT-2; Brown et al. (2020) GPT-3. For vision, Dosovitskiy et al. (2020) ViT. Tay et al. (2022) surveys efficient Transformers. The Illustrated Transformer (Jay Alammar) remains the best visual walkthrough.

7.13 Chapter Summary

Self-attention computes $QK^\top/\sqrt{d_k}$ scores, softmax, then $\times V$ – each token attends to all tokens in $O(1)$ steps, fully parallel. Multi-head gives diverse relations; positional encodings restore order. Pre-norm residual blocks scale to billions of parameters. BERT (masked, bidirectional) excels at understanding; GPT (causal, autoregressive) at generation. Attention is now universal across modalities.

Key takeaways: (1) Q, K, V are learned projections; (2) $\sqrt{d_k}$ prevents saturation; (3) h heads = parallel relations; (4) PE restores order; (5) $O(n^2)$ cost motivates sparse/Flash variants.

Scaling note. Transformers scale as $O(n^2 d)$ in sequence length. For $n > 2048$, use FlashAttention or sparse patterns (Longformer, BigBird) to stay within memory.

Positional pitfalls. Sinusoidal PE extrapolates, learned PE does not—interpolate or use RoPE/ALiBi for long contexts.

Head diversity. Visualize attention heads: early heads learn local syntax, deeper heads long-range semantics. If all heads look identical, increase h or add head dropout.

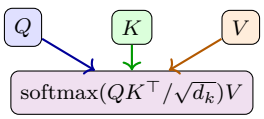


Figure 7.1: Self-attention as $QK^\top$ scoring then value weighting.

Example 7.2 (Attention scores)**.** For $d_k = 64$, $q = [0.2, -0.1, \dots]$, $k = [0.3, 0.4, \dots]$, dot $q^\top k \approx 2.1$, scaled $2.1/8 = 0.26$, softmax over 4 keys gives weights $[0.35, 0.28, 0.22, 0.15]$—soft selection, not hard.

7.14 Exercises

Exercise 7.1. Compute $QK^\top$ scaling: if $q, k \sim \mathcal{N}(0,1)^{d_k}$, what is $\mathrm{Var}[q^\top k]$? Why does $\sqrt{d_k}$ prevent softmax saturation?

Exercise 7.2. Show self-attention is permutation equivariant without PE: if you permute input rows, output permutes identically. Why does this force positional encodings?

Exercise 7.3. Count parameters for one Transformer layer with $d = 512$, $h = 8$, $d_{\text{ff}} = 2048$. Compare to a 3×3 conv with same d.

Exercise 7.4. Implement causal masking in NumPy self-attention: set scores for $j > i$ to $-\infty$ before softmax. Verify token i only attends to $\leq i$.

Exercise 7.5. Contrast BERT (masked LM, bidirectional) and GPT (causal, autoregressive) pre-training. Which suits classification vs generation? Can you convert one to the other?

Chapter 8

Training, Tuning & Deployment

Architecture is half the battle – the rest is making it actually work and stay working in the real world.

From zero: no MLOps background assumed. We start from a model that will not train and build to hyperparameters, debugging checklists, and deployment. Every recipe is motivated by a failure mode first; jargon is defined on first use.

Learning Outcomes

After this chapter you will be able to:

(L1) tune learning rate, batch size, and weight decay on a log scale;

(L2) apply a debugging checklist for stalled, diverged, or NaN training;

(L3) compare hyperparameter search strategies and ensure reproducibility;

(L4) describe deployment concerns: export, monitoring, and MLOps basics.

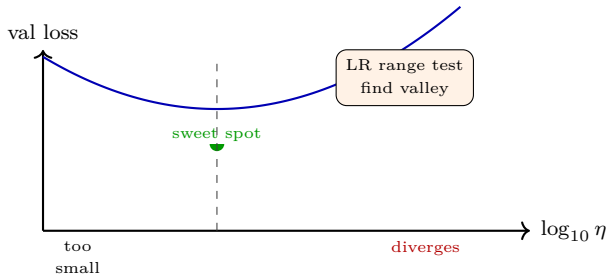
8.1 Intuition: From Toy to Production

You have a Transformer, a loss, and Adam. Yet training loss stalls, validation diverges, or gradients explode to NaN overnight. Real projects spend 80% of time on **hyperparameters, debugging, and deployment** – not model design. This chapter is the field manual.

8.2 Hyperparameters that Matter

- **Learning rate** η: often the single most important knob. Too high diverges, too low crawls. Search on log scale: 10^{-5} to 10^{-1}. Use LR range test – increase η exponentially for a few epochs, pick the steepest descent before divergence.

- **Batch size** B: 32–256 typical. Larger B gives stabler gradient but needs larger η (linear scaling rule: double B, double η with warmup). Small B regularizes via noise.
- **Weight decay** λ: 10^{-4}–10^{-2} for AdamW. Tune jointly with η.
- **Depth/width**: deeper = more capacity but harder to optimize (needs residuals + norm). Wider = more parallelism.
- **Regularization**: dropout 0.1–0.5, augmentation strength, label smoothing.

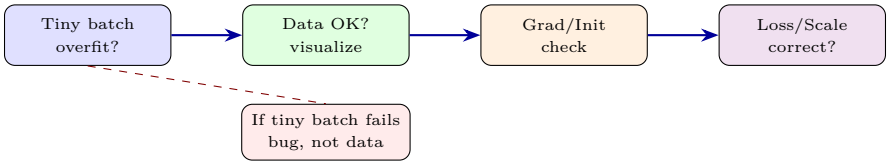


Systematic search: random search beats grid search in high dimensions (Bergstra & Bengio). For budgets, use **Bayesian optimization** or **Hyperband/ASHA** (early-stop bad trials). Always log every run (Weights & Biases, TensorBoard).

8.3 Debugging Checklist

When loss is NaN or flat, run through this order:

1. **Overfit a tiny batch**: train on 8 examples – if loss does not hit ≈ 0, bug exists (LR too low/high, wrong loss, frozen weights).
2. **Check data**: visualize augmented images, tokenization, label distribution. Leakage between train/val?
3. **Gradient norms**: log $\|\nabla\|$ per layer; explosion $\rightarrow$ clip; vanishing $\rightarrow$ check init/activation/norm.
4. **Initialization**: Xavier/He. $\mathrm{Var}[W] = 2/\text{fan\_in}$ for ReLU (He). Wrong init stalls deep nets.
5. **Loss scale**: cross-entropy expects logits (no softmax); MSE with unnormalized targets explodes.
6. **Determinism**: fix seeds, disable nondeterministic cuDNN for reproducibility.



8.4 Deployment: Beyond Accuracy

- **Quantization**: FP32 $\rightarrow$ INT8 halves memory, speeds 2–4$\times$ on edge. Calibrate ranges; use QAT (quantization-aware training) for $< 1\%$ drop.
- **Pruning & distillation**: remove 80% weights or distill large teacher $\rightarrow$ small student via soft targets $p_i^T = \text{softmax}(z_i/T)$.
- **Latency vs throughput**: batch-1 latency matters for interactive; batch-64 throughput for offline. Profile with TensorRT/ONNX.
- **Monitoring**: track input drift (PSI, embedding distance), prediction distribution, and human-evaluated quality. Retrain triggers when drift exceeds threshold.
- **Ethics & safety**: bias audits, adversarial robustness, privacy (DP-SGD), and explainability (SHAP, attention maps).

8.5 Worked Example: Tuning and Export

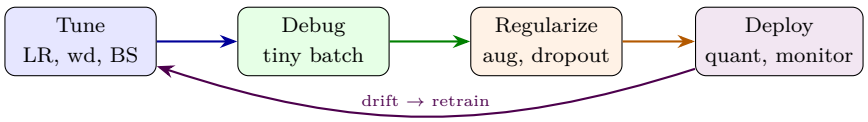
Example 8.1 (Hyperparameter Search + ONNX Export).

```
import torch, torch.nn as nn, optuna, onnx
# 1. LR range test
model = nn.Sequential(nn.Linear(64,64), nn.ReLU(), nn.Linear(64,10))
opt = torch.optim.SGD(model.parameters(), lr=1e-7)
lrs, losses = [], []
for lr_mult in [1.2]*100:
    for g in opt.param_groups: g['lr']*=lr_mult
    x,y = next(iter(train_loader))
    loss = nn.CrossEntropyLoss()(model(x), y)
    loss.backward(); opt.step(); opt.zero_grad()
    lrs.append(opt.param_groups[0]['lr']); losses.append(loss.item())

# 2. Optuna search (random/Bayesian)
def objective(trial):
    lr = trial.suggest_float("lr",1e-5,1e-1,log=True)
    wd = trial.suggest_float("wd",1e-6,1e-2,log=True)
    p  = trial.suggest_float("dropout",0.0,0.5)
    # build & train 3 epochs, return val acc
    acc = train_and_eval(lr, wd, p)  # user-defined
    return acc
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)
print("Best:", study.best_params)

# 3. Export to ONNX for deployment
dummy = torch.randn(1,64)
torch.onnx.export(model, dummy, "model.onnx", input_names=["input"], output_names=["logits
    "])
# Quantize / serve with onnxruntime or TensorRT
import onnxruntime as ort
sess = ort.InferenceSession("model.onnx")
print(sess.run(None, {"input": dummy.numpy()})[0].shape)
```

Monitoring snippet:

```python
# Drift detection: compare embedding means
import numpy as np
ref_mean = np.load("ref_emb_mean.npy")  # from training
live_emb = model.encode(live_batch)     # current serving
drift = np.linalg.norm(live_emb.mean(0)-ref_mean)
if drift > 0.3:
    print(f"ALERT drift={drift:.2f} -> trigger retrain")
```

8.6 Putting It All Together



8.7 Hyperparameter Search Strategies Compared

Grid search scales as k^d (exponential in dims) and wastes trials on irrelevant dims. Random search is embarrassingly parallel and finds good regions faster (Bergstra & Bengio, 2012). Bayesian optimization (GP-UCB, TPE) models $f(\lambda)$ and explores where expected improvement is high – 2–5× more sample-efficient for expensive training. Hyperband/ASHA early-stops poor trials, allocating budget to promising ones; combined as BOHB. Always report search budget and best-vs-final variance.

8.8 Initialization and Normalization Deep Dive

He init assumes ReLU; for GELU/SiLU use $\approx 2/\text{fan}_{in}$ as well, but gain differs. Fixup and T-Fixup initialize deep residual nets without Norm by scaling residual branches by $L^{-1/2}$. LayerNorm normalizes over features per token (good for variable length); BatchNorm over batch (good for vision, fails for small B or sequence). RMSNorm drops mean-centering for speed. Choose based on modality and batch regime – mismatching causes instability.

8.9 Debugging Case Studies

Case A: loss flat at init – check last-layer bias for class imbalance, or softmax+cross-entropy double-softmax bug. Case B: NaN at step 500 – usually FP16 overflow or log(0); add $\epsilon = 10^{-7}$, use BF16, clip logits. Case C: train acc 95%, val 60% – overfit; add augmentation, dropout, wd, or more data; check for leakage (duplicates across splits). Case D: val loss oscillates – LR too high or batch too small; add warmup, increase B, or use EMA of weights (Polyak averaging) for smoother eval.

8.10 Deployment Pipeline and MLOps

A typical pipeline: train → validate → export ONNX → quantize (INT8, FP16) → benchmark (latency, memory, accuracy) → canary deploy (5% traffic) → monitor (drift, latency P99, error rate) → rollback or promote. Use feature stores for consistent train/serve features, and model registry for versioning. For LLMs, add KV-cache, continuous batching (vLLM), and speculative decoding for throughput. Cost-aware autoscaling on GPU utilization keeps spend predictable.

8.11 Reproducibility and Ethics Checklist

Fix all seeds (Python, NumPy, PyTorch, CUDA), set `CUBLAS_WORKSPACE_CONFIG`, and log code+data hashes. Version datasets with DVC. For ethics: measure subgroup performance (gender, locale), test adversarial robustness (FGSM, AutoAttack), apply DP-SGD if privacy-sensitive, and document limitations in a model card. No model is "done" at deployment – it enters a lifecycle of measurement, mitigation, and update.

8.12 Common Pitfalls

Tuning on test data inflates reported accuracy – keep test locked. Changing two hyperparameters at once confounds attribution; use ablations. Not logging random seeds makes "improvements" irreproducible. Overlooking data pipeline bottlenecks (CPU augmentation, I/O) leaves GPUs idle – profile with `torch.utils.bottleneck`. Finally, optimizing accuracy alone hides latency/cost: a 0.5% gain that doubles inference cost may not ship.

8.13 Further Reading

Bergstra & Bengio (2012) random search; Li et al. (2018) Hyperband; Snoek et al. Bayesian optimization. For deployment, see Huyen, *Designing ML Systems*, and PyTorch's quantization guide. For MLOps, explore Weights & Biases reports and the ONNX/TensorRT documentation – the fastest way to cut latency in practice.

8.14 Chapter Summary

Tuning, debugging, and deployment decide real success. LR η matters most – find it with range tests and schedules. Tiny-batch overfitting finds bugs fast; gradient norms, init, and loss scales are the usual suspects. Quantize, prune, or distill for deployment; monitor drift and canary before full rollout. Reproducibility (seeds, hashes) and ethics (bias, privacy) are non-optional.

Key takeaways: (1) log-scale search for η, λ; (2) random > grid; Hyperband prunes early; (3) tiny-batch overfit = smoke test; (4) INT8 + ONNX/TensorRT for serving; (5) drift > threshold → retrain.

8.15 Exercises

Exercise 8.1. Run an LR range test on a 3-layer MLP on MNIST: sweep $\eta = 10^{-6}$ to 10^{-1} exponentially for one epoch, plot loss vs $\log \eta$, and pick the optimal η.

Exercise 8.2. Your loss is NaN after epoch 2. List three likely causes and the log/diagnostic that would reveal each (gradient norm, weight histogram, input stats).

Exercise 8.3. Compare INT8 post-training quantization vs QAT on TinyCNN/CIFAR-10. Report accuracy drop and model size. When does QAT matter most?

Exercise 8.4. Design a Bayesian hyperparameter search space for a Transformer (LR, warmup, wd, dropout, heads). Justify log vs linear scales.

Exercise 8.5. For a production image classifier, propose a monitoring dashboard: three metrics for data drift, three for model health, and a retraining trigger policy.